Lab Assignment-2

School of Engineering and Technology
Course Name & Code: Operating System Lab ENCA252 Program Name: BCA (AI & DS) (Research)

Problem Statement

Deadlock is a situation in operating systems where a set of processes are unable to proceed because each is waiting for resources held by others. Banker's Algorithm is a deadlock avoidance technique that ensures the system remains in a safe state. In this assignment, students will implement Banker's Algorithm and determine whether the system is in a safe state, as well as find the safe sequence of execution.

Learning Objectives

• Understand CPU scheduling concepts
• Implement FCFS and SJF algorithms
• Calculate CT, TAT, WT
• Analyze system performance

Tools/Technology Used:

• Python 3.x
• Built-in Python libraries: os, multiprocessing, time
• Linux OS (optional for simulation)

Assignment Tasks:

Task 1: System Input and Data Representation Objective:

To understand how resources are allocated to processes in an operating system.

Description:

Students will input system data including number of processes, number of resources, allocation matrix, maximum matrix, and available resources.

Sub-Tasks:

1. Define number of processes and resources.
2. Input Allocation Matrix.

3. Input Maximum Matrix.
4. Input Available Resources.

Expected Outcome:

· Understanding of system resource allocation
· Ability to represent system state **Task**

2: Need Matrix Calculation

Objective:

To compute the remaining resource requirements for each process.

Description:

Need Matrix is calculated using the formula:

Need = Maximum – Allocation Sub-Tasks:

1. Calculate Need Matrix.
2. Display the Need Matrix.

Expected Outcome:

· Clear understanding of process requirements
· Ability to derive required resources

Task 3: Banker's Safety Algorithm Implementation

Objective:

To determine whether the system is in a safe state.

Description:

The safety algorithm checks if there exists a safe sequence in which all processes can execute without causing deadlock.

Sub-Tasks:

1. Initialize Work = Available.
2. Initialize Finish array for all processes.
3. Find a process whose Need ≤ Work.
4. Allocate resources and update Work.

5. Repeat until all processes are completed or no further allocation is possible.

Expected Outcome:

· Ability to check safe/unsafe state
· Understanding of deadlock avoidance

<u>Task 4: Safe Sequence Determination</u>

Objective:

To find

Rubrics for Evaluation

Criterion	**Weight**	**Excellent (5)**	**Good (4)**	**Fair (3) or Poor (2 or less)**
Implementation & Execution	40%	Correct simulation of all components	Minor logic or output issues	Significant errors or missing outputs
Code Quality & Documentation	30%	Well-commented and structured code	Mostly readable, few comments	Poor formatting or no documentation
Output Accuracy	20%	All outputs correct and validated	Minor output mismatches	Incomplete or incorrect output
Report & Submission	10%	Submitted on time, clear report	Minor issues or delay	Late or no report

CODE

```
# Banker's Algorithm Implementation
ì
def calculate_need(max_matrix, allocation, n, m):
    need = [[0]*m for _ in range(n)]
    for i in range(n):
        for j in range(m):
            need[i][j] = max_matrix[i][j] - allocation[i][j]
    return need


def is_safe(n, m, allocation, max_matrix, available):
    need = calculate_need(max_matrix, allocation, n, m)

    print("\nNeed Matrix:")
    for row in need:
        print(row)

    work = available[:]
    finish = [False] * n
    safe_sequence = []

    while len(safe_sequence) < n:
        found = False
        for i in range(n):
            if not finish[i]:
                if all(need[i][j] <= work[j] for j in range(m)):

                    # Add allocated resources back
                    for j in range(m):
                        work[j] += allocation[i][j]

                    safe_sequence.append(i)
                    finish[i] = True
                    found = True
```

```
        if not found:
            return False, []

    return True, safe_sequence


# ----------- MAIN PROGRAM -------------

n = int(input("Enter number of processes: "))
m = int(input("Enter number of resources: "))

print("\nEnter Allocation Matrix:")
allocation = []
for i in range(n):
    allocation.append(list(map(int, input().split())))

print("\nEnter Maximum Matrix:")
max_matrix = []
for i in range(n):
    max_matrix.append(list(map(int, input().split())))

print("\nEnter Available Resources:")
available = list(map(int, input().split()))

# Safety Check
safe, sequence = is_safe(n, m, allocation, max_matrix, available)

if safe:
    print("\nSystem is in SAFE state.")
    print("Safe Sequence:", " -> ".join(f"P{i}" for i in sequence))
else:
    print("\nSystem is in UNSAFE state (Deadlock possible).")
```

REPORT

Title:

Implementation of Banker's Algorithm for Deadlock Avoidance

Objective:

To implement Banker's Algorithm and check whether the system is in a safe state.

Theory:

Banker's Algorithm is used in operating systems to avoid deadlock. It checks whether resources can be allocated safely without causing deadlock.

Formula Used:

Need = Maximum – Allocation

Steps Performed:

1. Enter number of processes and resources
2. Input Allocation Matrix
3. Input Maximum Matrix
4. Calculate Need Matrix
5. Apply Banker's Safety Algorithm
6. Determine Safe Sequence

Output:

The program checks whether the system is in a safe state and displays the safe sequence

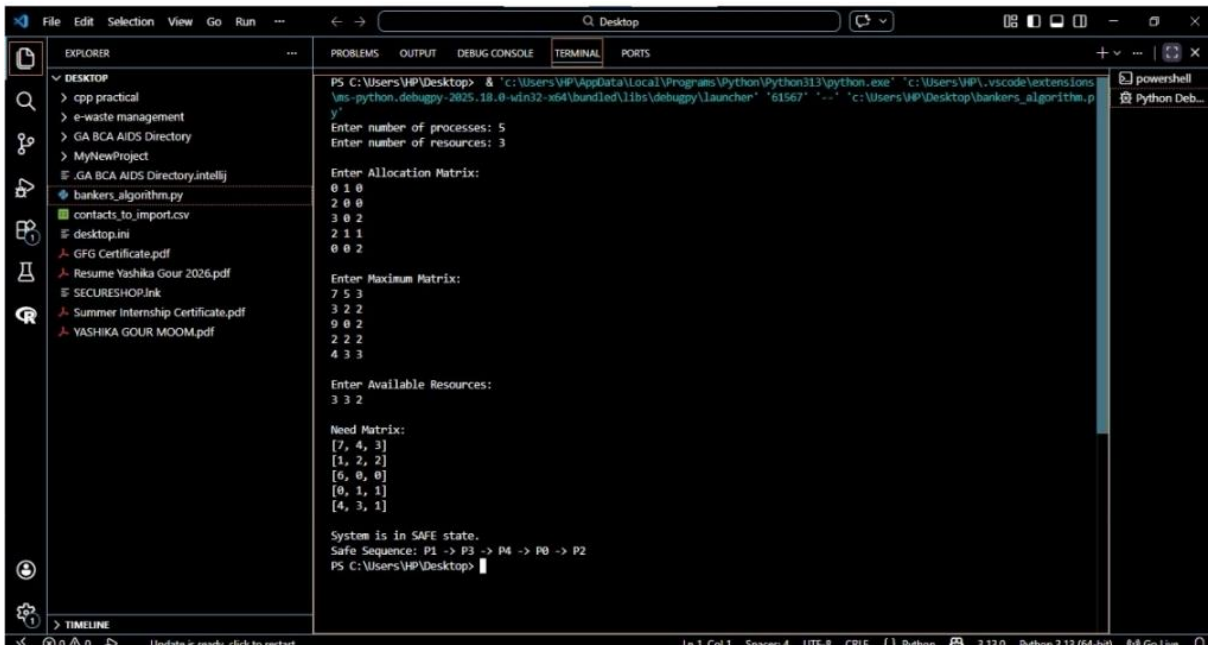
Result:

The system is in a SAFE state and safe sequence is displayed.

Conclusion:

The Banker's Algorithm successfully avoids deadlock and ensures safe execution of processes.

OUTPUT


File Edit Selection View Go Run
Desktop
EXPLORER
DESKTOP
cpp practical
e-waste management
GA BCA AIDS Directory
MyNewProject
.GA BCA AIDS Directory.intellij
bankers_algorithm.py
contacts_to_import.csv
desktop.ini
GFG Certificate.pdf
Resume Yashika Gour 2026.pdf
SECURESHOP.lnk
Summer Internship Certificate.pdf
YASHIKA GOUR MOOM.pdf
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
Python Deb...
PS C:\Users\HP\Desktop> & 'c:\Users\HP\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\HP\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61567' '--' 'c:\Users\HP\Desktop\bankers_algorithm.py'
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
Need Matrix:
[7, 4, 3]
[1, 2, 2]
[6, 0, 0]
[0, 1, 1]
[4, 3, 1]
System is in SAFE state.
Safe Sequence: P1 -> P3 -> P4 -> P0 -> P2
PS C:\Users\HP\Desktop>